

PP23 Final Project

Parallel Ant Colony Optimization for TSP

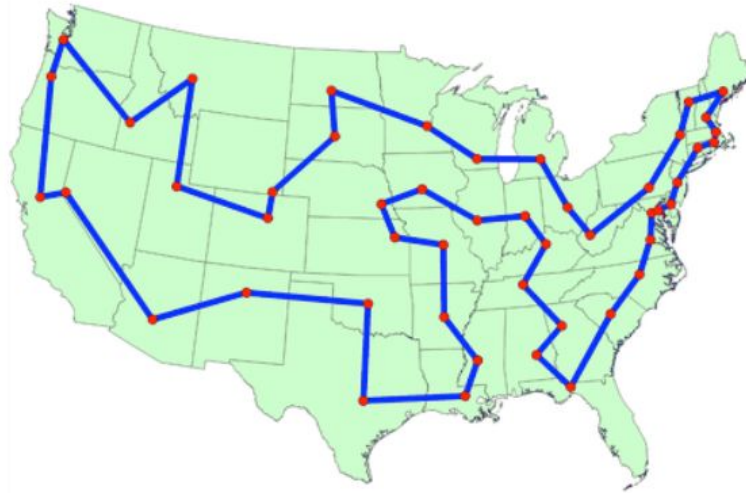
109062315 洪聖祥
109062317 許淳

Outline

- Problem Description: TSP
- Ant Colony Optimization
- Dataset : TSPLIB
- CPU Parallelization
- GPU Parallelization
- ACO Algorithm Result
- Profiling Result
- Reference

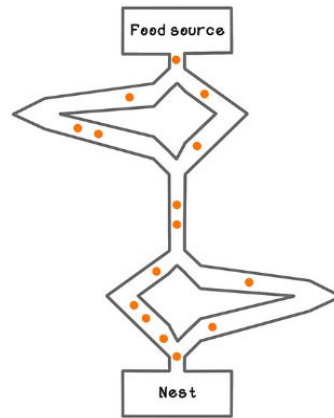
Problem Description: TSP

- Visit all cities at once
- The tour forms a hamilton cycle

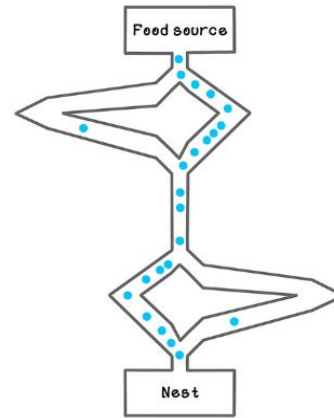


Ant Colony Optimization (ACO)

- A meta-heuristic algorithm which applies to many problems including scheduling problem, routing problem...
- Consists of 3 stages Initialization, tour construction and Pheromone update
- Iterate tour construction and pheromone update stage until convergence or set a MAX_ITER



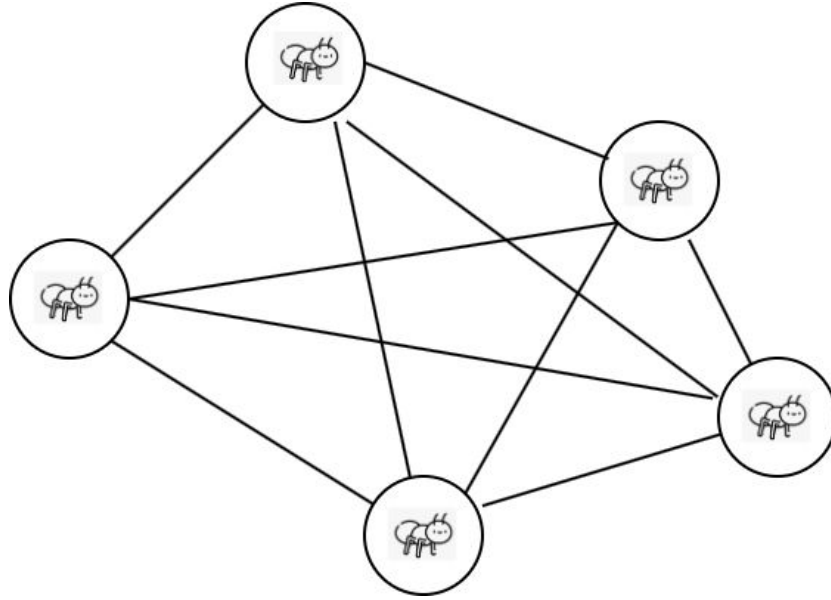
After 4 minutes



After 8 minutes

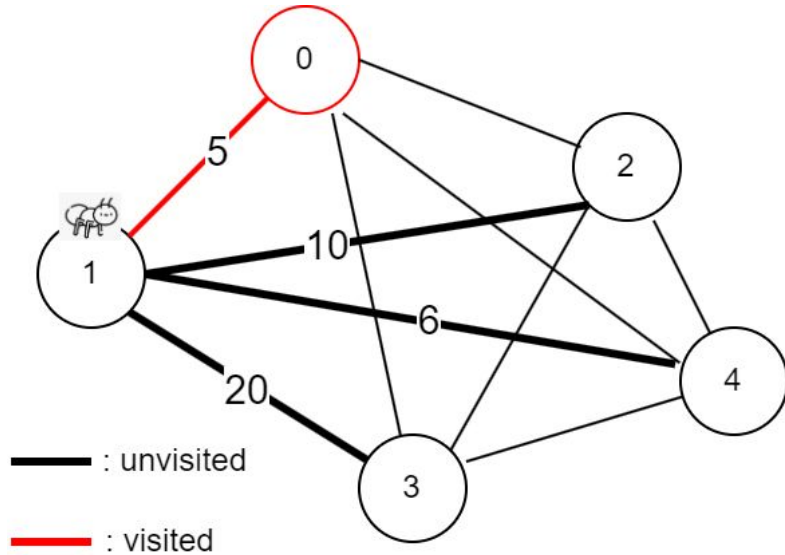
Ant Colony Optimization - Initialization

Initialize one ant at each city



Ant Colony Optimization - Tour Construction

An ant select next city based on the fitness of the edge (fitness is based on pheromone quantity)

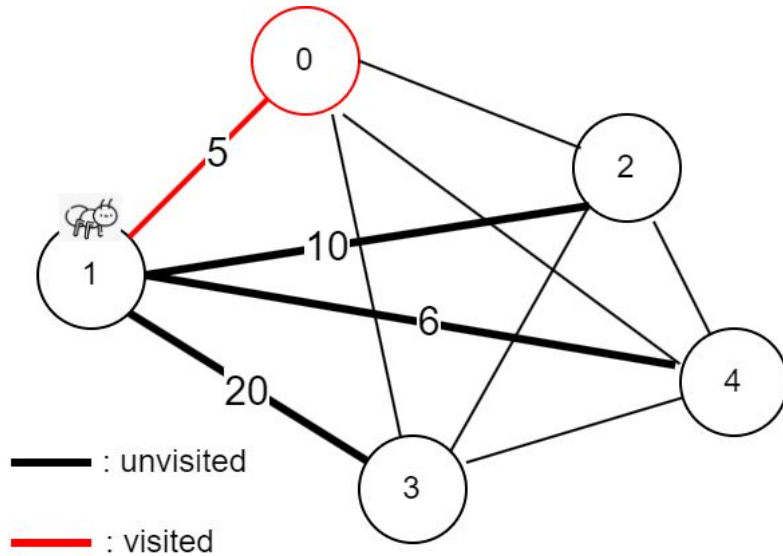


Probability of selecting next city

City	0	1	2	3	4
Pr	0	0	10/36	6/36	20/36

Ant Colony Optimization - Tour Construction

Randomly choose a number between 0-1 and select next city based on the random number



Normalized prefix sum of pheromone

0	1	2	3	4
0	0	10/36	30/36	36/36

Generate random number (0-1): 0.5

Find i s.t $\text{prefix}[i-1] < 0.5 \leq \text{prefix}[i]$
 $\Rightarrow i = 3$

Ant Colony Optimization - Pheromone Update

Pheromone Evaporation

$$\tau(i, j) \leftarrow (1 - \rho)\tau(i, j) \quad \tau(i, j) : \text{how much pheromone on edge (i,j)}$$

Pheromone Update based on every ant's tour distance

$$\tau(i, j) \leftarrow \tau(i, j) + \sum_{k=1}^m \Delta\tau_k(i, j)$$

$$\Delta\tau_k(i, j) = \begin{cases} \frac{1}{C_k} & \text{if } e_{i,j} \in T_k \\ 0 & \text{otherwise,} \end{cases} \quad C_k : \text{tour distance constructed by ant k}$$

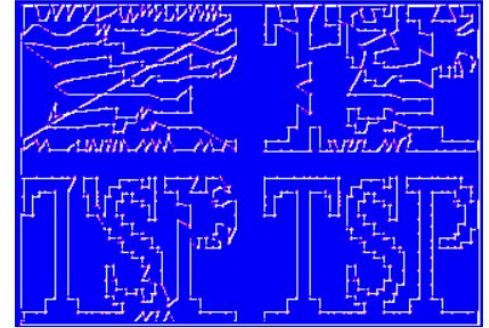
Dataset : TSPLIB

TSPLIB is a library of sample instances for the TSP (and related problems) from various sources and of various types.

We choose the symmetric TSP dataset from TSPLIB

Choose 6 datasets for profiling

ch130	a280	lin318	pcb442	rat783	pr1002
-------	------	--------	--------	--------	--------



Link: [TSPLIB95](http://www.tsp95.com)

CPU parallelization

- OpenMP
- Tour construction of each ant is independent, so it can be parallelized
- If multiple threads attempt to update the pheromone level on the same path at the same time, it can lead to race conditions. And updating pheromone is not expensive, so we don't parallelized it.

Experiment

- Environment: Apollo
- Test cases from small to large

testcase (#cities)	Sequential [s]	12 core [s]	Speed-up
ch130	1.332	0.17	7.835294118
a280	11.345	1.348	8.416172107
lin318	16.891	1.862	9.071428571
pcb442	45.281	4.57	9.908315098
rat783	251.289	24.577	10.22455955
pr1002	511.395	49.22	10.38998375

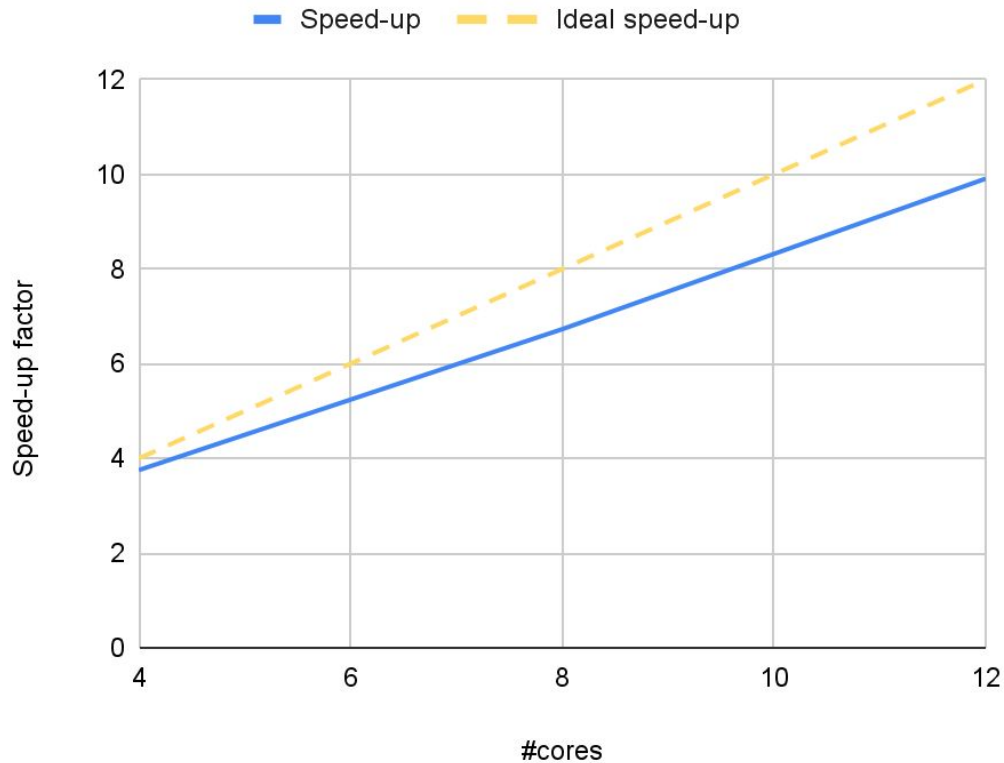
Experiment

Profiling on the smallest test case

#cores	1	4	8	12
Initialize	0.001076462	0.001487043	0.001308405	0.001576418
Tour construction	1.330412762	0.354766442	0.228039838	0.173142376
Update pheromone	0.000759236	0.000875597	0.001269936	0.001275929
Free resources	0.000008152	0.000014424	0.000022521	0.001576418
Total	1.332256612	0.357143506	0.2306407	0.177571141
Speed-up	1	3.730311736	5.776329208	7.502664028

Strong Scalability

Testcase: pcb442



CUDA Parallelization

- ACO includes many 2D matrices which includes element-wise operation
- Next city selection : Parallel prefix sum

CUDA Parallelization : Element-wise operation

2D pheromone matrix:

$$\tau(i, j) \leftarrow \tau(i, j) + \sum_{k=1}^m \Delta\tau_k(i, j)$$

2D fitness matrix:

$$f(i, j) = [\tau(i, j)]^\alpha [\eta(i, j)]^\beta,$$

CUDA Parallelization: Parallel Prefix Sum

- Referenced from [Nvidia Developers](#)

Including three Optimizations:

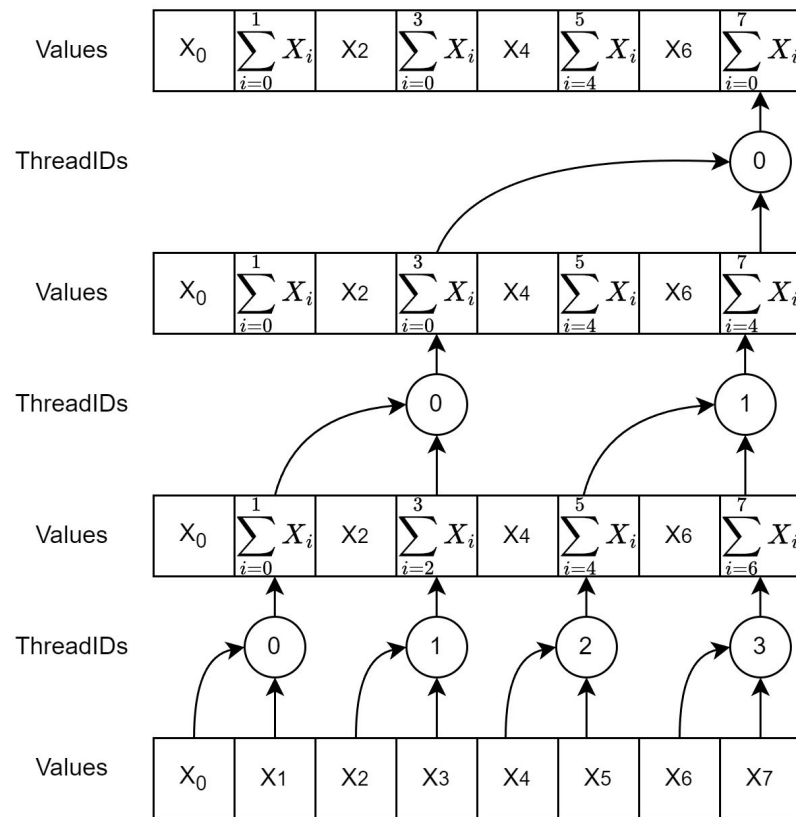
- Make use of shared memory
- Avoiding warp divergence
- Avoiding bank conflict

Parallel Prefix Sum : Up Sweep

Load all values to shared memory

Perform a similar operation of All reduction and maintain a tree-like structure

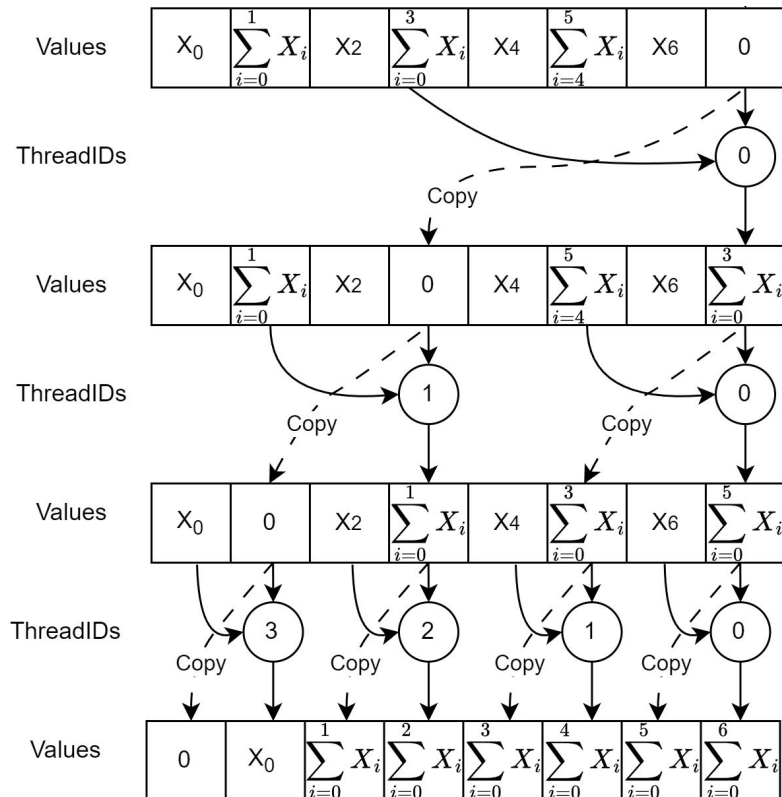
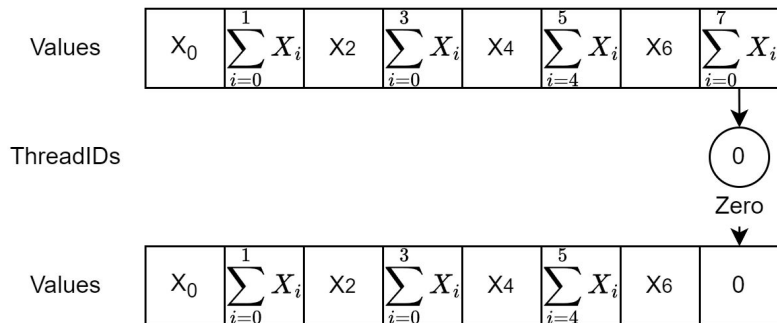
Avoid warp divergence by restricting threadID in every iteration



Parallel Prefix Sum : Down Sweep

Inserting zero at the root of the tree

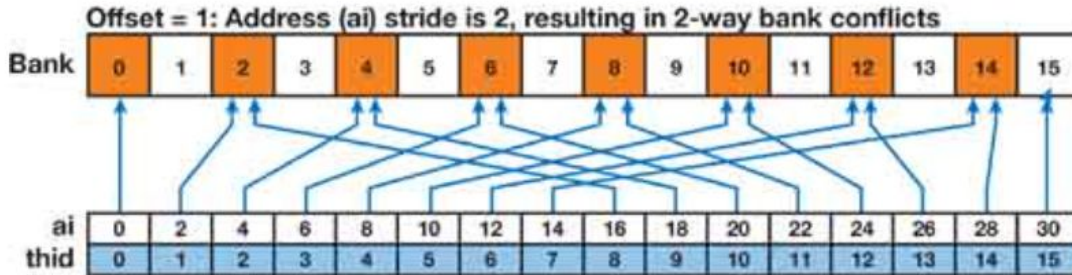
On each step, each node at the current level passes its own value to its left child, and the sum of its value and the former value of its left child to its right child.



Parallel Prefix Sum : Avoid Bank Conflict

Addressing Without Padding

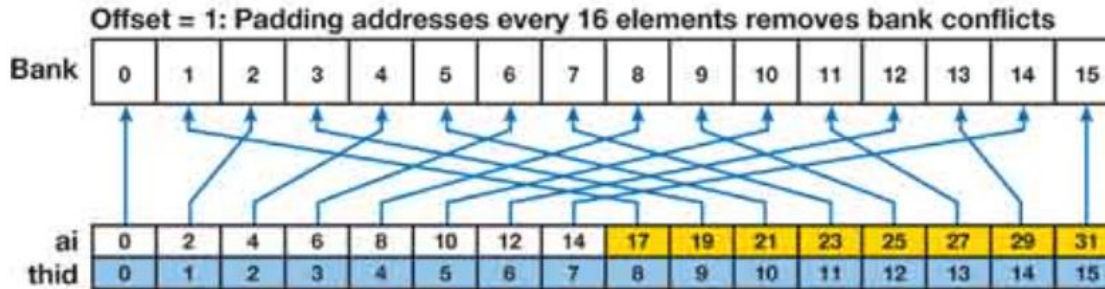
```
int ai = offset*(2*thid+1)-1;  
int bi = offset*(2*thid+2)-1;  
temp[bi] += temp[ai]
```



Parallel Prefix Sum : Avoid Bank Conflict

```
int ai = offset*(2*thid+1)-1;
int bi = offset*(2*thid+2)-1;
ai += ai / NUM_BANKS;
bi += bi / NUM_BANKS;
temp[bi] += temp[ai]
```

Addressing With Padding



Apply Prefix sum on Selecting next city

Profile on pr1002 and set 1 iteration of ACO algo

Naive approach: iterate through all ants and calculate their prefix sum

Time(%)	Time	Calls	Avg	Min	Max	Name
98.00%	5.22090s	1003002	5.2050us	4.6400us	18.112us	prescan_arbitrary(float*, int, int)

Parallel approach: parallel all ants prefix sum operation

Time(%)	Time	Calls	Avg	Min	Max	Name
48.05%	96.570ms	1001	96.473us	91.393us	102.53us	prescan_arbitrary(float*, int, int)

ACO Result

We compare the tour distance constructed by greedy and ACO algo

	ch130	a280	lin318	pcb442	rat783	pr1002
Greedy Distance	7575	3148	54033	61984	11255	315596
ACO Distance	6489	2806	47867	58838	10480	298156
Optimal Distance	6110	2579	42029	50778	8806	259045

The result of the tour by ACO algo can be downloaded in [google drive](#)

GPU Profiling Result

Testcases (#cities)	Sequential (s)	GPU (s)	Speedup
ch130	1.332	0.224088	5.94
a280	11.345	0.300855	37.7
lin318	16.891	0.296659	56.93
pcb442	45.281	0.482048	93.93
rat783	251.289	2.164158	116.11
pr1002	511.395	3.720389	137.45

References

ACO algorithm:

https://en.wikipedia.org/wiki/Ant_colony_optimization_algorithms

ACO parallel implementation:

https://www.cs.hiroshima-u.ac.jp/cs/_media/4893a094.pdf

GPU prefix sum implementation:

<https://developer.nvidia.com/gpugems/gpugems3/part-vi-gpu-computing/chapter-3-9-parallel-prefix-sum-scan-cuda>

TSPLIB dataset:

<http://comopt.ifi.uni-heidelberg.de/software/TSPLIB95/tsp/>